

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

LABORATÓRIO DE CIRCUITOS

Projeto Integrador

Subsistema de Memória e Unidade de Controle

Turma do 3º semestre • Simulação em Logisim-Evolution

Peso total: 100 pontos • Parte I (50%) + Parte II (50%)

Campo	Preenchimento
Professor responsável	Prof. Dr. Herbert Oliveira Rocha
Semestre letivo	2026.2
Modalidade	Trabalho em equipe (até dois alunos)
Integrante 1	_____
Integrante 2	_____
Repositório GitHub	_____

1. Apresentação e regras gerais

Esta atividade substitui a lista de dezessete componentes isolados do Laboratório de Circuitos. A avaliação recai sobre o subsistema montado, e não sobre a peça isolada.

Parte	Objeto	Artefatos exigidos
Parte I (50%)	Módulo de memória de 64 bytes com decodificação de endereços e uma cache de 4 linhas medida sobre um traço de leituras	Circuito funcional (.circ) + planilha de acertos e faltas
Parte II (50%)	Unidade de controle cabeada de 5 estados e o microprograma equivalente	Projeto simulado (.circ) + tabela de tempo + microprograma

Tabela 1 — Estrutura de avaliação.

1.1. Equipes, prazos e integridade

- Todos devem ter commits próprios no repositório.
- Atrasos: desconto de 10% da nota por dia corrido, até o limite de 3 dias.
- É permitido consultar bibliografia, documentação e colegas de outras equipes para esclarecer conceitos. É vedado compartilhar arquivos .circ, planilhas ou trechos de relatório entre equipes.
- O uso de ferramentas de IA generativa é permitido desde que declarado em seção própria do relatório, informando quais ferramentas, em quais etapas e com que finalidade. Haverá arguição individual: quem não souber explicar o próprio circuito perde a nota do trabalho.

2. Objetivos de aprendizagem

Nº	Nível (Bloom)	Objetivo
OA1	Compreender	Explicar por que vários dispositivos podem compartilhar um mesmo barramento e qual é o papel do decodificador de endereços nesse arranjo.
OA2	Aplicar	Montar, em Logisim-Evolution, um módulo de memória de 64 bytes com quatro regiões selecionadas por um decodificador 2→4.
OA3	Analisar	Decompor endereços de 6 bits em rótulo, índice e deslocamento e classificar cada acesso de um traço como acerto, falta compulsória ou falta por conflito.
OA4	Aplicar	Calcular a taxa de acertos e o tempo médio de acesso à memória a partir da planilha do traço.
OA5	Aplicar	Implementar uma máquina de estados de cinco estados que gere os sinais de controle de um caminho de dados multiciclo.
OA6	Analisar	Minimizar por mapas de Karnaugh as funções de próximo estado e derivar as equações booleanas correspondentes.
OA7	Criar	Escrever o microprograma de oito bits equivalente à unidade de controle cabeada e compará-lo com ela.

Tabela 2 — Objetivos de aprendizagem.

3. Pré-requisitos e ferramentas

Pressupõe-se domínio de álgebra booleana, mapas de Karnaugh, circuitos combinacionais e flip-flops, conteúdos de Circuitos Digitais. Conceitos de cache e de unidade de controle são apresentados neste enunciado e nas aulas, e não precisam ser conhecidos de antemão.

Ferramenta	Versão de referência	Uso
Logisim-Evolution	3.8.0 ou superior	Montagem e simulação.
Planilha eletrônica	Calc, Excel ou Google Sheets	Planilha (modelo disponibilizado em aula) de acertos e faltas e cálculo dos indicadores.
Git e GitHub	—	Controle de versão e entrega.

Tabela 3 — Ferramentas exigidas.

Este documento descreve o que cada circuito deve fazer, não onde clicar no simulador. Localizar os componentes na biblioteca faz parte do trabalho.

4. Onde entra cada componente

A tabela abaixo indica, para cada um dos dezessete componentes, em qual parte ele é usado e qual função exerce. Nenhum deles pode ser entregue como circuito avulso: a correção verifica se está instanciado e conectado dentro do subsistema correspondente.

Nº	Componente	Parte	Função no subsistema
01	Flip-flop D e flip-flop JK	I e II	Registrador de estado da unidade de controle (3 flip-flops D) e bits de validade das 4 linhas da cache.
02	Multiplexador de 4 entradas	I e II	Seleção da saída de dados entre os quatro dispositivos de memória; seleção da segunda entrada da ULA.
03	XOR a partir de AND, NOT e OR	I	Comparador de rótulos de 3 bits da cache.
04	Somador de 8 bits com constante 4	II	Incremento do contador de programa no ciclo de busca.
05	Memória ROM de 8 bits	I e II	Memória de programa da região 0x00–0x0F (Parte I) e memória de microcódigo de 8 posições (Parte II).
06	Memória RAM de 8 bits	I	Memória de dados da região 0x10–0x1F e arranjo de dados da cache.
07	Banco de registradores de 8 bits	I e II	Região 0x20–0x2F do mapa de memória e leitura dos operandos do caminho de dados.
08	Somador de 8 bits	I	Cálculo do endereço a ser apresentado à cache.
09	Detector da sequência "101"	II	Exercício preparatório de máquina de estados, entregue antes da unidade de controle.
10	ULA de 8 bits	II	Execução das operações comandadas pelo sinal ALUOp.
11	Extensor de sinal de 4 para 8 bits	II	Tratamento do deslocamento da instrução de desvio.
12	Máquina de estados com portas lógicas	II	Unidade de controle cabeada de cinco estados.
13	Contador síncrono	II	Microcontador de programa (μ PC) de 3 bits.
14	Detector de paridade ímpar	I	Bit de paridade das palavras gravadas na RAM.

Nº	Componente	Parte	Função no subsistema
15	Otimização por mapas de Karnaugh	II	Minimização das funções de próximo estado e dos sinais de controle.
16	Decodificador de 7 segmentos	I e II	Exibição do estado atual da máquina e do contador de acertos da cache.
17	Detector de número primo (4 bits)	I	Geração de quatro endereços adicionais para o teste de conflito da Seção 5.6.

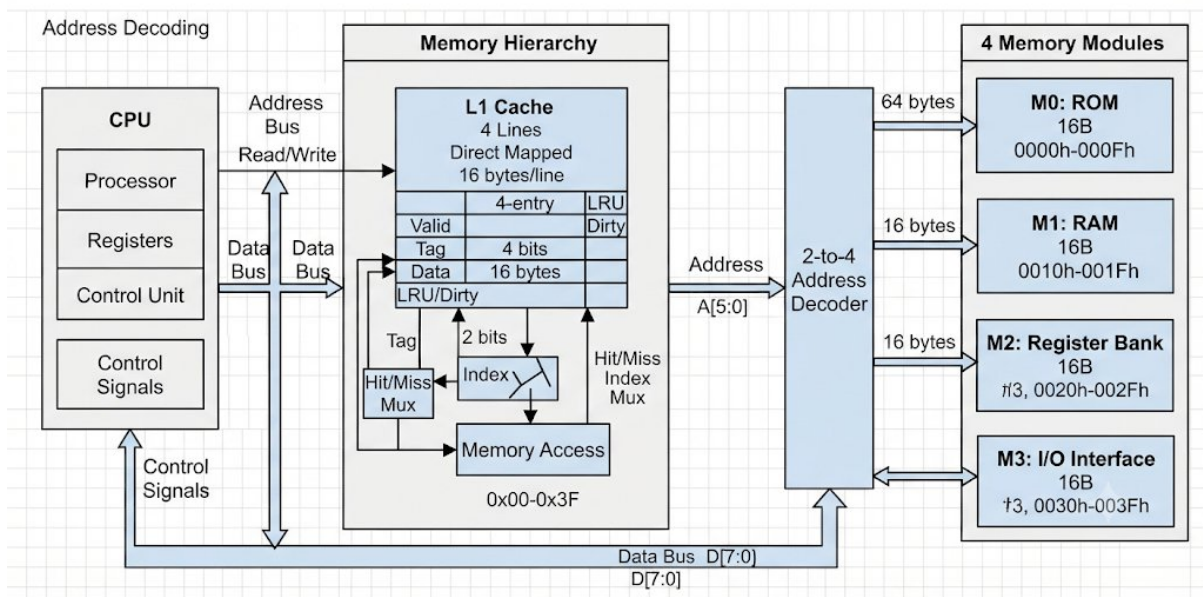
Tabela 4 — Função de cada componente nos dois subsistemas.

5. Parte I — Subsistema de memória (50 pontos)

5.1. O que será construído

Um espaço de memória de 64 bytes, dividido em quatro regiões de 16 bytes, com um decodificador escolhendo qual delas responde a cada endereço. Sobre esse módulo, uma cache pequena, de quatro linhas, cujo comportamento será medido acesso a acesso.

DIAGRAMA DE BLOCOS: ARQUITETURA DE MEMÓRIA E CACHE



5.2. Módulo de memória

- Barramento de endereços de 6 bits (A5..A0), totalizando 64 posições.
- Barramento de dados de 8 bits.
- Os dois bits mais significativos, A5 e A4, entram em um decodificador 2→4 que escolhe a região.
- Os quatro bits restantes, A3..A0, endereçam uma das 16 posições internas da região escolhida.

Faixa	Dispositivo	Tamanho	Sinal de seleção
0x00 – 0x0F	ROM de programa	16 bytes	CS_ROM = A5' · A4'

Faixa	Dispositivo	Tamanho	Sinal de seleção
0x10 – 0x1F	RAM de dados	16 bytes	$CS_RAM = A5' \cdot A4$
0x20 – 0x2F	Banco de registradores	16 bytes	$CS_REG = A5 \cdot A4'$
0x30 – 0x3F	Região de E/S	16 bytes	$CS_IO = A5 \cdot A4$

Tabela 5 — Mapa de memória e equações de seleção.

A equipe deve montar a tabela verdade de A5 e A4 mostrando que exatamente um dos quatro sinais CS fica ativo em cada combinação. Essa tabela tem apenas quatro linhas e deve constar do relatório.

Para juntar as saídas dos quatro dispositivos, use o multiplexador de 4 entradas do Componente 02, comandado pelos mesmos bits A5 e A4. Uma implementação com buffers de três estados também é aceita e vale bonificação (Seção 8.1), mas não é exigida nesta versão da atividade.

5.2.1. Bit de paridade

Toda palavra gravada na RAM deve ter um bit de paridade ímpar associado, gerado pelo Componente 14 no momento da escrita e recalculado na leitura. Se os dois valores divergirem, um sinal de erro deve acender no circuito. Como a RAM é de 8 bits, mantenha os bits de paridade em um arranjo separado de 16 posições de 1 bit.

5.3. A cache

A cache fica entre a CPU e o módulo de memória. Nesta atividade ela atende somente leituras, o que dispensa qualquer discussão de política de escrita. Seus parâmetros são fixos:

Parâmetro	Valor
Organização	Mapeamento direto
Número de linhas	4
Tamanho do bloco	2 bytes
Capacidade de dados	8 bytes
Metadados por linha	1 bit de validade + 3 bits de rótulo
Operações atendidas	Somente leitura
Tempo de acerto	1 ciclo
Penalidade de falta	8 ciclos

Tabela 6 — Parâmetros da cache.

5.3.1. Como o endereço se divide

Cada endereço de 6 bits se parte em três campos. A equipe deve reproduzir esta demonstração no relatório, explicando de onde vem cada número:

```

deslocamento = log2(tamanho do bloco) = log2(2) = 1 bit    -> A0
índice        = log2(numero de linhas) = log2(4) = 2 bits   -> A2 A1
rótulo        = 6 - 2 - 1 = 3 bits                          -> A5 A4 A3

```

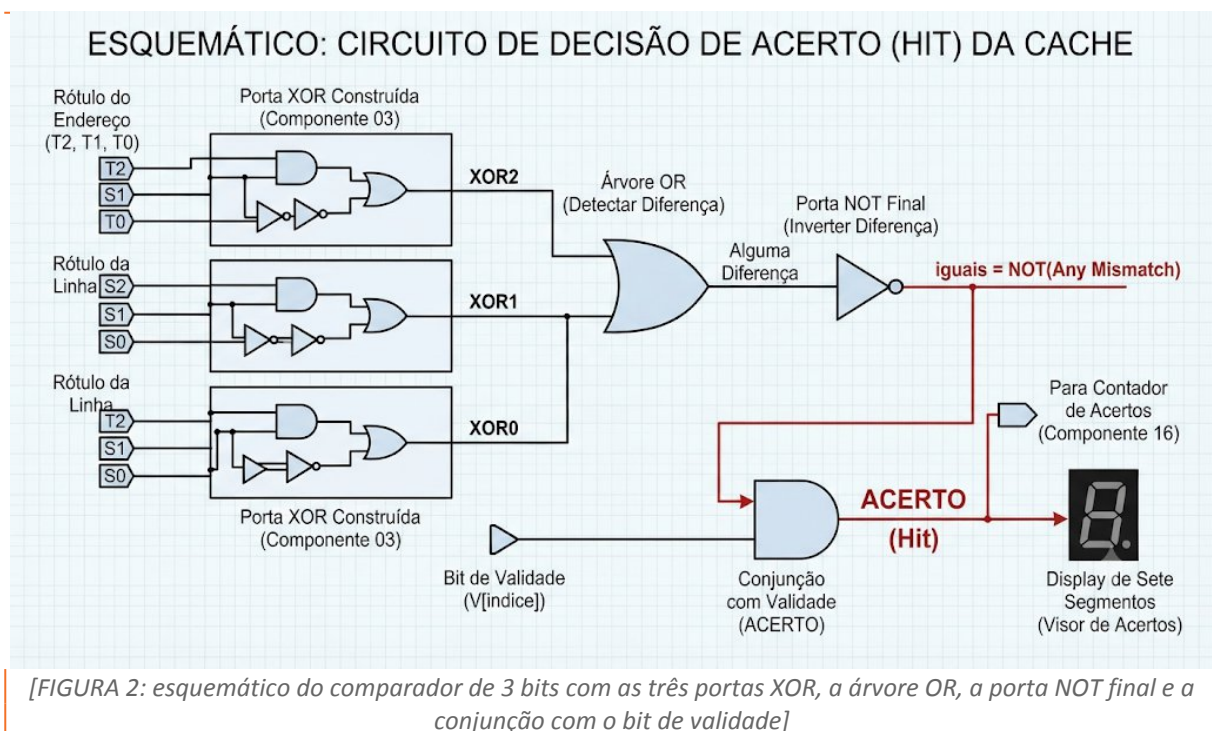
Exemplo: o endereço 0x22 vale 100010 em binário. Separando da direita para a esquerda, o deslocamento é 0, o índice é 01 e o rótulo é 100. Portanto esse endereço só pode ocupar a linha 1 da cache, e ali será registrado o rótulo 100.

5.3.2. Como o circuito decide se houve acerto

A cache compara o rótulo do endereço pedido com o rótulo guardado na linha indicada pelo índice. A comparação deve ser feita com portas XOR construídas a partir de AND, NOT e OR (Componente 03), e não com um comparador pronto da biblioteca:

```
iguais = NOT( (T2 XOR S2) OR (T1 XOR S1) OR (T0 XOR S0) )
ACERTO = iguais AND valido[indice]
```

T2..T0 é o rótulo do endereço pedido e S2..S0 é o rótulo guardado na linha. O bit de validade indica se aquela linha já foi preenchida alguma vez. Um contador de acertos deve alimentar o Componente 16 e aparecer em um display de sete segmentos.



5.4. O traço de acessos

O traço abaixo tem acessos, todos de leitura, e é obrigatório. Ele foi montado para que metade resulte em acerto e metade em falta, de modo que os dois casos apareçam com clareza.

Nº	End.	Op	Nº	End.	Op	Nº	End.	Op	Nº	End.	Op
1	0x00	L	5	0x00	L	9	0x04	L	13	0x22	L
2	0x01	L	6	0x20	L	10	0x05	L	14	0x23	L
3	0x02	L	7	0x21	L	11	0x06	L	15	0x02	L
4	0x03	L	8	0x00	L	12	0x07	L	16	0x03	L

Tabela 7 — Traço obrigatório de leituras.

5.5. A planilha

A planilha é entregue como arquivo separado e tem uma linha por acesso. As colunas são obrigatórias e nesta ordem:

Col.	Cabeçalho	Conteúdo
A	acesso	Número do acesso.
B	endereço_hex	Endereço em hexadecimal.
C	endereço_bin	Endereço em binário, seis dígitos.
D	rotulo	Rótulo, 3 bits.
E	índice	Índice, 2 bits.
F	deslocamento	Deslocamento, 1 bit.
G	valido_antes	Bit de validade da linha indexada, antes do acesso.
H	rotulo_armazenado_antes	Rótulo que estava na linha, antes do acesso, ou traço se vazia.
I	resultado	ACERTO ou FALTA.
J	tipo_falta	compulsoria, conflito ou vazio.
K	rotulo_apos	Rótulo que passa a ocupar a linha após o acesso.

Tabela 8 — Colunas obrigatórias da planilha.

As colunas C, D, E, F, I e J devem ser obtidas por fórmula a partir da coluna B, e não digitadas uma a uma. A correção confere a presença das fórmulas.

5.5.1. Exemplo resolvido

Os restantes são responsabilidade da equipe.

Ac.	Hex	Binário	Rótulo	Índ.	Desl.	Válido antes	Resultado	Por quê
1	0x00	000000	000	00	0	0	FALTA	Linha 0 vazia: falta compulsória.

Tabela 9 — Cinco primeiros acessos resolvidos.

5.5.2. Indicadores

Com a planilha pronta, calcule e apresente, mostrando as expressões usadas:

1. Número de acertos, número de faltas e taxa de acertos.
2. Quantas faltas foram compulsórias e quantas foram por conflito.
3. O tempo médio de acesso à memória, pela expressão $AMAT = \text{tempo de acerto} + (\text{taxa de faltas} \times \text{penalidade de falta})$, com os valores da Tabela 6.

5.6. Teste de conflito com o detector de primos

Este teste usa o Componente 17 e mostra o principal defeito de uma cache de mapeamento direto. Percorra os valores $v = 0$ a 7 em um contador de 3 bits. Para cada v que o detector indicar como primo, gere um acesso de leitura ao endereço obtido deslocando v três posições à esquerda. Como os primos entre 0 e 7 são 2, 3, 5 e 7, resultam quatro acessos:

v	Endereço	Binário	Rótulo	Índice	Resultado esperado
2	0x10	010000	010	00	FALTA

v	Endereço	Binário	Rótulo	Índice	Resultado esperado
3	0x18	011000	011	00	FALTA
5	0x28	101000	101	00	FALTA
7	0x38	111000	111	00	FALTA

Tabela 10 — Teste de conflito. Comece com a cache invalidada.

Os quatro endereços têm índice 00 e rótulos diferentes: todos disputam a mesma linha e nenhum acerto acontece, mesmo havendo três linhas ociosas. Explique no relatório por que a regra de geração produz esse resultado e o que aconteceria se a cache tivesse oito linhas em vez de quatro.

5.7. Entregáveis da Parte I

Entregável	Critério de aceitação
parte1_memoria.circ	Abre sem erros; cada região responde na faixa correta; MUX de saída comandado por A5 e A4; sinal de erro de paridade demonstrável.
parte1_cache.circ	Comparador de rótulos feito com XOR sintetizado; bits de validade zeráveis; contador de acertos em display de sete segmentos.
parte1_planilha.xlsx	16 linhas, onze colunas na ordem pedida, colunas derivadas por fórmula, indicadores da Seção 5.5.2 e o teste da Seção 5.6.
Seção do relatório	Mapa de memória, tabela verdade dos CS, demonstração da divisão do endereço, indicadores e evidências de teste.

Tabela 11 — Entregáveis da Parte I.

6. Parte II — Unidade de controle (50 pontos)

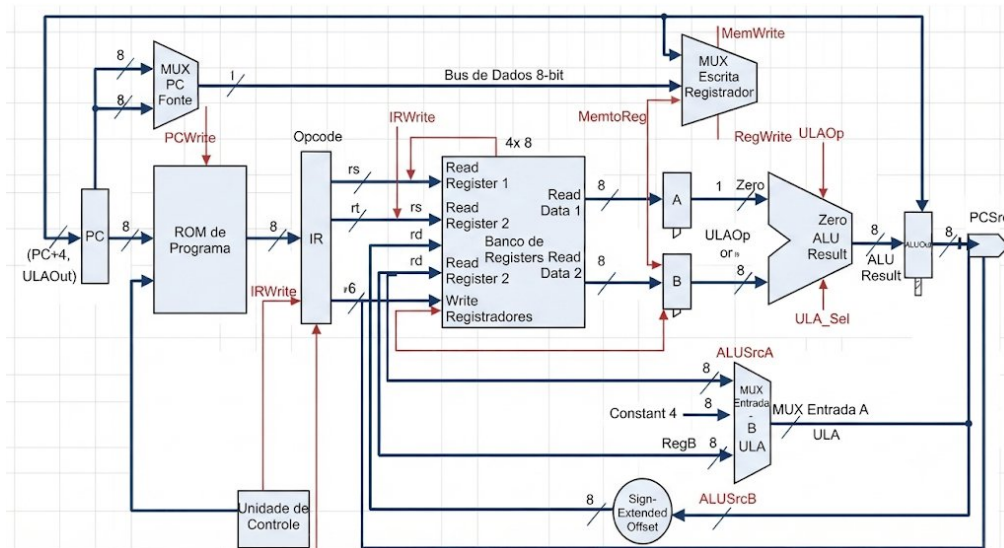
6.1. O caminho de dados

O caminho de dados é fornecido no estilo MIPS multiciclo, reduzido a 8 bits. A equipe o monta a partir de componentes que já construiu; o que se projeta aqui é a unidade de controle que o comanda.

Nesta versão o conjunto de instruções é curto e não inclui acesso a dados na memória, o que dispensa os registradores MDR e o multiplexador de origem do endereço:

Instrução	Tipo	Semântica	Componentes envolvidos
add	R	$rd \leftarrow rs + rt$	07, 10
sub	R	$rd \leftarrow rs - rt$	07, 10
and	R	$rd \leftarrow rs \text{ AND } rt$	07, 10
or	R	$rd \leftarrow rs \text{ OR } rt$	07, 10
beq	I	se $rs = rt$, $PC \leftarrow PC + 4 + (\text{ext}(\text{imm}) \ll 2)$	04, 10, 11

Tabela 12 — Instruções suportadas. As quatro primeiras formam uma única classe do ponto de vista do controle.



[FIGURA 3: caminho de dados multiciclo de 8 bits, com PC, ROM de programa, IR, banco de registradores, registradores A e B, ULA, ULAOut e os multiplexadores, com os sinais de controle rotulados]

6.2. Sinais de controle

São nove sinais, somando onze bits.

Sinal	Bits	Função
PCWrite	1	Escreve no PC incondicionalmente.
PCWriteCond	1	Escreve no PC apenas se a ULA indicar Zero.
PCSource	1	Origem do valor do PC: 0 = saída da ULA, 1 = ULAOut.
MemRead	1	Habilita a leitura da ROM de programa.
IRWrite	1	Carrega o registrador de instrução.
RegWrite	1	Habilita a escrita no banco de registradores.
ALUSrcA	1	Primeira entrada da ULA: 0 = PC, 1 = registrador A.
ALUSrcB	2	Segunda entrada: 00 = registrador B, 01 = constante 4, 10 = extensão de sinal deslocada de 2.
ALUOp	2	Operação: 00 = soma, 01 = subtração, 10 = definida pelo campo funct.

Tabela 13 — Sinais de controle.

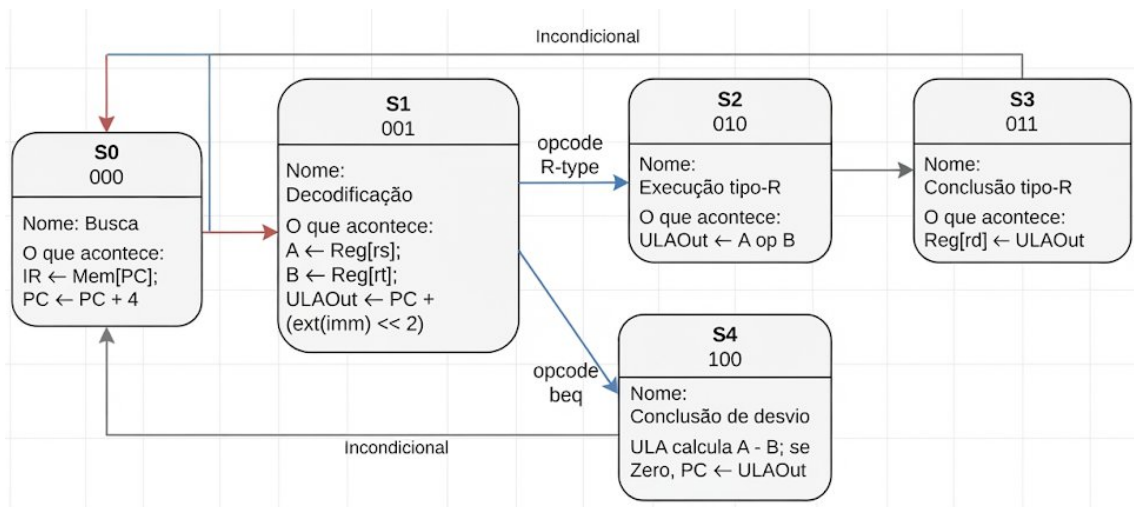
6.3. A máquina de estados

São exatamente cinco estados, o que exige três flip-flops D (Componente 01). Restam três combinações não utilizadas — 101, 110 e 111 — cujo tratamento a equipe deve escolher e justificar: tratá-las como condições indiferentes, ou forçar o retorno ao estado de busca.

Estado	Código	Nome	O que acontece
S0	000	Busca	$IR \leftarrow \text{Mem}[PC]; PC \leftarrow PC + 4$
S1	001	Decodificação	$A \leftarrow \text{Reg}[rs]; B \leftarrow \text{Reg}[rt]; \text{ULAOut} \leftarrow PC + (\text{ext}(\text{imm}) \ll 2)$
S2	010	Execução tipo-R	$\text{ULAOut} \leftarrow A \text{ op } B$
S3	011	Conclusão tipo-R	$\text{Reg}[rd] \leftarrow \text{ULAOut}$
S4	100	Conclusão de desvio	ULA calcula $A - B$; se Zero, $PC \leftarrow \text{ULAOut}$

Tabela 14 — Os cinco estados da unidade de controle.

As transições são simples: S0 sempre vai para S1. De S1, o opcode decide: instrução tipo-R vai para S2, instrução beq vai para S4. S2 vai para S3. S3 e S4 voltam para S0.



[FIGURA 4: diagrama de estados com os cinco estados, a bifurcação em S1 conforme o opcode e o retorno de S3 e S4 para S0]

6.4. Unidade de controle cabeada

O que deve constar do relatório:

- Diagrama de estados completo, com as condições de transição escritas em função do opcode.
- Tabela de transição, com estado atual em 3 bits, entradas, estado seguinte e saídas. São cinco linhas úteis.
- Tabela verdade dos nove sinais de controle em função do estado. Como cada sinal depende só do estado, cada tabela tem cinco linhas.
- Mapas de Karnaugh das três funções de próximo estado e de, no mínimo, três sinais de controle, com os agrupamentos desenhados.
- Equações booleanas finais, em soma de produtos, para os sinais e para as três entradas dos flip-flops D.
- Circuito no Logisim-Evolution usando apenas portas AND, OR e NOT e flip-flops D. Nesta versão é vedado usar ROM ou tabela de verdade da biblioteca para gerar os sinais.
- Contagem do número de portas usadas.

O estado atual deve aparecer em um display de sete segmentos (Componente 16), o que permite acompanhar a execução ciclo a ciclo.

6.5. Tabela de tempo

Monte uma tabela com os estados nas linhas e os nove sinais nas colunas, marcando 0, 1, o valor binário do campo, ou X quando o valor for indiferente. Cada X deve ser justificado. O primeiro estado já vem preenchido como referência.

Estado	PCWr	PCWrC	PCSrc	MemRd	IRWr	RegWr	SrcA	SrcB	ALUOp
S0 (Busca)	1	0	0	1	1	0	0	01	00
S1 (Decod.)	0	0	X	0	0	0	0	10	00
S2 ... S4	—	—	—	—	—	—	—	—	—

Tabela 15 — Modelo de tabela de tempo. SrcA e SrcB abreviam ALUSrcA e ALUSrcB.

6.5.1. Ciclos e CPI

Pela tabela de tempo, determine quantos ciclos consome cada classe de instrução e calcule o CPI médio para a mistura abaixo, apresentando a conta:

Classe	Frequência	Estados percorridos
Tipo-R (add, sub, and, or)	70%	S0, S1, S2, S3
beq	30%	S0, S1, S4

Tabela 16 — Mistura de instruções para o cálculo do CPI.

6.6. O microprograma equivalente

A segunda implementação faz exatamente a mesma coisa que a primeira, mas guardando os sinais em uma memória em vez de gerá-los com portas. O sequenciamento fica a cargo de um microcontador de programa de 3 bits (Componente 13), e os sinais ficam em uma ROM de 8 posições por 8 bits (Componente 05).

6.6.1. Formato da microinstrução

A microinstrução tem exatamente oito bits, divididos em quatro campos de dois bits. Isso é o que permite armazená-la em uma única ROM de 8 bits.

Ordem	Campo	Bits	Codificação
1	ULAOp	2	00 = soma; 01 = subtração; 10 = operação dada pelo campo funct; 11 = reservado
2	Fonte	2	00 = PC e constante 4; 01 = registrador A e registrador B; 10 = PC e extensão de sinal deslocada de 2; 11 = reservado
3	Ação	2	00 = nenhuma; 01 = $IR \leftarrow Mem[PC]$; 10 = $Reg[rd] \leftarrow ULAOut$; 11 = $PC \leftarrow ULAOut$ se Zero
4	Sequenciamento	2	00 = próxima microinstrução; 01 = volta à busca; 10 = despacho pelo opcode; 11 = reservado

Tabela 17 — Formato da microinstrução de 8 bits.

Convenção adotada para caber em oito bits: o campo Fonte com valor 00 implica, além de selecionar PC e a constante 4 como entradas da ULA, a escrita incondicional do resultado no PC. É por isso que a microinstrução de busca não precisa de um campo próprio de controle do PC.

6.6.2. Duas microinstruções resolvidas

A microinstrução de busca precisa carregar o IR e incrementar o PC ao mesmo tempo. Concatenando os quatro campos na ordem da Tabela 17:

```
Busca:  ULAOp=00  Fonte=00  Acao=01  Seq=00
        00 | 00 | 01 | 00  -> 00000100 = 0x04
```

A de decodificação lê os registradores e adianta o cálculo do destino do desvio, entregando o controle ao despacho pelo opcode:

```
Decod:  ULAOp=00  Fonte=10  Acao=00  Seq=10
        00 | 10 | 00 | 10 -> 00100010 = 0x22
```

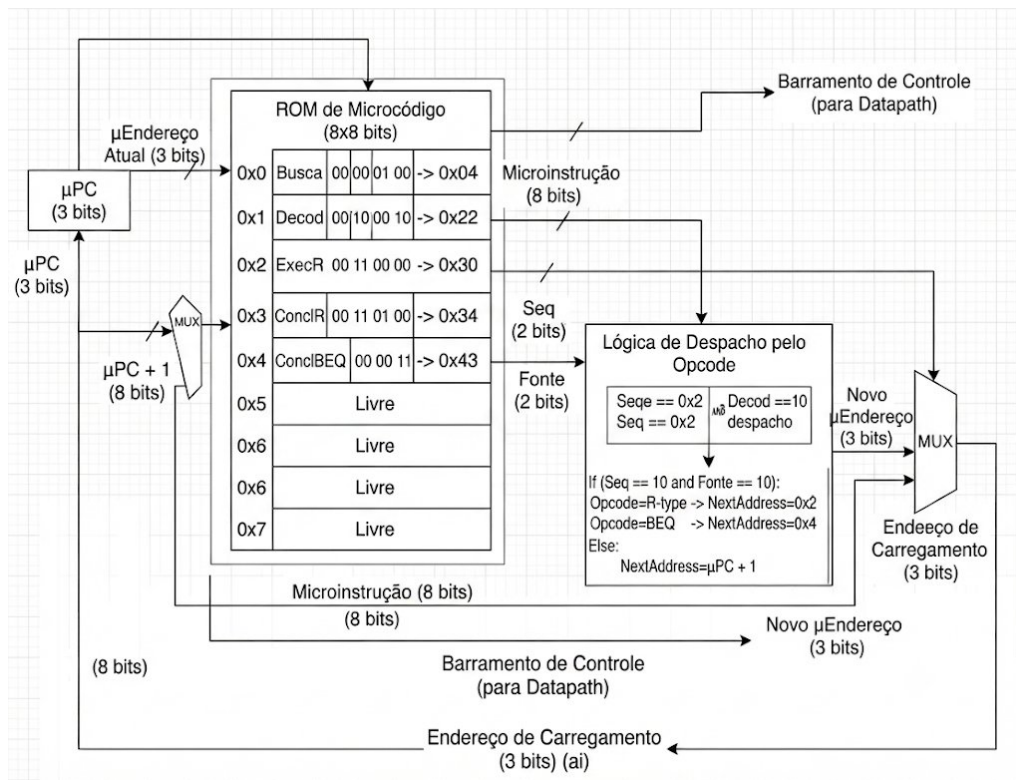
6.6.3. O que a equipe deve entregar

Complete a tabela do microprograma com as três microinstruções restantes — as de execução tipo-R, conclusão tipo-R e conclusão de desvio — no formato abaixo, e carregue o conteúdo resultante na ROM de microcódigo.

Rótulo	Endereço	ULAOp	Fonte	Ação	Seq	Hexadecimal
Busca	0	00	00	01	00	0x04
Decod	1	00	10	00	10	0x22
ExecR	2					
ConclR	3					
ConclBEQ	4					

Tabela 18 — Tabela do microprograma a completar. As posições 5, 6 e 7 da ROM ficam livres.

A lógica de despacho é simples: ao chegar em Decod com Seq igual a 10, o opcode escolhe entre o endereço 2, para instruções tipo-R, e o endereço 4, para beq. Entregue também o conteúdo da ROM em arquivo de texto carregável pelo simulador, chamado microcodigo.txt.



[FIGURA 5: diagrama da unidade de controle microprogramada, com μPC de 3 bits, ROM de microcódigo, lógica de despacho pelo opcode e o multiplexador que escolhe a próxima microinstrução]

6.7. Comparação entre as duas versões

Escreva de 300 a 400 palavras comparando as duas implementações que a própria equipe construiu, com números medidos e não com afirmações genéricas de livro. Responda a estas quatro perguntas:

- Quantas portas lógicas a versão cabeada consumiu, e quantos bits de ROM a versão microprogramada consumiu?
- Qual das duas foi mais rápida de projetar, e por quê?
- Se fosse preciso acrescentar uma quinta operação à ULA, quantas equações teriam de ser refeitas na versão cabeada, e quantas microinstruções teriam de ser escritas na versão microprogramada?
- Em que situação de projeto real cada uma das duas abordagens seria preferível?

6.8. Entregáveis da Parte II

Entregável	Critério de aceitação
parte2_cabeada.circ	Cinco estados com três flip-flops D e lógica minimizada; sem ROM nem tabela de verdade da biblioteca; estado visível em sete segmentos.
parte2_microprogramada.circ	μPC de 3 bits funcional, ROM de microcódigo carregada, despacho pelo opcode operante.
microcodigo.txt	Conteúdo da ROM coerente com a Tabela 18 completada.
tabela_tempo.xlsx ou .pdf	Cinco estados, nove colunas de sinais, condições indiferentes justificadas, cálculo do CPI.
Seção do relatório	Diagrama de estados, tabela de transição, mapas de Karnaugh desenhados, equações finais, tabela do microprograma e a comparação da Seção 6.7.

Tabela 19 — Entregáveis da Parte II.

7. Testes obrigatórios

Cada circuito precisa vir acompanhado de evidência de que funciona. Para cada caso abaixo, registre os pinos de entrada acionados, o valor observado nos pinos de saída e uma captura de tela do simulador com os valores legíveis.

ID	O que se verifica	Entradas	Saída esperada	Result.
T-01	Seleção de região	A5A4 = 00, 01, 10 e 11	Um único CS ativo em cada caso	☐
T-02	Erro de paridade	Gravar 0x55, inverter um bit, ler	Sinal de erro aceso	☐
T-03	Falta compulsória	Cache zerada, ler 0x00	ACERTO = 0	☐
T-04	Acerto no mesmo bloco	Depois de T-03, ler 0x01	ACERTO = 1	☐
T-05	Falta por conflito	Depois de T-03, ler 0x20	ACERTO = 0 e rótulo da linha 0 passa a 100	☐
T-06	Ciclo de busca	Reset e um pulso de relógio	Estado vai de S0 para S1; IRWrite pulsado	☐
T-07	Execução de uma instrução tipo-R	add na ROM de programa	Quatro ciclos; rd recebe o resultado	☐
T-08	Desvio tomado e não tomado	beq com operandos iguais e depois diferentes	PC muda apenas no primeiro caso	☐

ID	O que se verifica	Entradas	Saída esperada	Result.
T-09	Equivalência das duas versões	Mesmo programa nas duas	Mesma sequência de sinais, ciclo a ciclo	☐

Tabela 20 — Casos de teste obrigatórios.

Circuito entregue sem evidência de teste é avaliado como não funcional, mesmo que o arquivo .circ esteja correto.

8. Como o trabalho será avaliado

Nível	Significado
Insuficiente	Ausente, não funcional ou conceitualmente errado.
Suficiente	Presente e parcialmente funcional, com erros conceituais ou documentação incompleta.
Bom	Completo e funcional, com falhas menores de precisão ou justificativa.
Excelente	Completo, funcional, justificado e com evidência de teste.

Tabela 21 — Níveis de desempenho.

9. Como entregar

A entrega é feita por repositório público no GitHub, com o nome no padrão abaixo, usando os primeiros nomes dos integrantes sem espaços e sem acentos:

AOC_Nome1Nome2_UFRR_LabCircuitos_2026

A estrutura de diretórios é obrigatória:

```
AOC_Nome1Nome2_UFRR_LabCircuitos_2026/
├── README.md
├── relatorio/relatorio_tecnico.pdf
├── parte1/
│   ├── parte1_memoria.circ
│   ├── parte1_cache.circ
│   ├── parte1_planilha.xlsx
│   └── evidencias/
├── parte2/
│   ├── parte2_cabeada.circ
│   ├── parte2_microprogramada.circ
│   ├── microcodigo.txt
│   ├── tabela_tempo.xlsx
│   └── evidencias/
└── componentes/
```

9.1. O que o README.md deve conter

11. Disciplina, semestre e integrantes com matrícula.
12. Versão do Logisim-Evolution usada e como abrir cada arquivo .circ.
13. Lista dos arquivos entregues, com uma linha de descrição para cada.

14. Em qual arquivo e em qual subcircuito cada um dos dezessete componentes foi instanciado.
15. Declaração de uso de ferramentas de IA generativa e divisão do trabalho entre os integrantes.

9.2. Estrutura do relatório

Introdução com os objetivos; uma seção por parte, seguindo a ordem deste enunciado; para cada componente, a descrição dos pinos, a imagem do circuito e os testes realizados com pinos de entrada, conexões ativas e resultado nas saídas; os indicadores da Parte I; a comparação da Parte II; conclusão e referências. Numere e referencie no texto todas as figuras e tabelas.